



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, <u>KMP algorithm</u> , Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours



BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module 3: String Matching Algorithms(5 Hours)

- Naive String Matching Algorithm
- KMP—Knuth Morris Pratt Algorithm
- **Rabin- Karp Algorithm**
- Suffix Trees



String Matching

- Text-editing programs frequently need to find all occurrences of a pattern in the text.
- Typically, the text is a document being edited, and the pattern searched for is a particular word supplied by the user.
- Efficient algorithms for this problem called **“String Matching”**
- **Example:**
 - Search for particular patterns in DNA sequences
 - Internet search engines – find Web pages relevant to queries.



Rabin Karp Algorithm

- The Rabin-Karp string searching algorithm calculates a **hash value** for the pattern, and for each M-character subsequence of text to be compared.
- If the **hash values are unequal**, the algorithm will calculate the hash value for next M-character sequence.
- If the **hash values are equal**, the algorithm will do a **Brute Force** comparison between the pattern and the M-character sequence.
- In this way, there is **only one comparison per text subsequence**, and Brute Force is only needed when hash values match.



Rabin Karp Algorithm

- Assign Numeric Codes to the characters in the text and pattern
- Ideally ASCII codes can be used.
- Find a single value for the pattern (called hash code) using a hash function
- Find hash codes in the text for piece of pattern length
 - If hash codes match, check individual characters, else take next piece of character.



Basic idea

Size=6

TEXT

a	a	a	a	a	b
1	2	3	4	5	6

PATTERN

Size=3

a	a	b
1	2	3

$$1 + 1 + 2 = 4$$

*Hash
Function*

*Hash
code*

Finding Single Value of the Pattern ??

Rolling Hash Function

$$O(n-m+1)$$

a	1
b	2
c	3
d	4
e	5

Drawback

TEXT

Size=11

c	c	a	c	c	a	a	e	d	b	a
1	2	3	4	5	6	7	8	9	10	11

PATTERN

Size=3

d	b	a
1	2	3

Finding Single Value of the Pattern ??

Hash Code= 4+2+1= 7

a	1
b	2
c	3
d	4
e	5
f	6
g	7
h	8
i	9
j	10

?????

Spurious Hits

$O(mn)$



Rabin Karp Algorithm

TEXT

Size=11

c	c	a	c	c	a	a	e	d	b	a
1	2	3	4	5	6	7	8	9	10	11

PATTERN

Size=3

d	b	a
1	2	3

Have to use **Strong Hash function**
to avoid spurious hits – **Given by Rabin Karp**

$$= 4 * 10^2 + 2 * 10^1 + 1 * 10^0$$

$$= P[1] * 10^{m-1} + P[2] * 10^{m-2} + P[3] * 10^{m-3}$$

$$= 421 \rightarrow \text{Hash code}$$

Average case: $O(n-m+1)$

Worst Case: $O(mn)$

Base
Value

a	1
b	2
c	3
d	4
e	5
f	6
g	7
h	8
i	9
j	10

Rabin Karp Algorithm

TEXT

Size=11

c	c	a	c	c	a	a	e	d	b	a
1	2	3	4	5	6	7	8	9	10	11

$$= 3 \cdot 10^2 + 3 \cdot 10^1 + 1 \cdot 10^0$$

= 331 - Not match so move on

$$[[3 \cdot 10^2 + 3 \cdot 10^1 + 1 \cdot 10^0] - 3 \cdot 10^2] \cdot 10 + 3 \cdot 10^0$$



Rolling Hash
Function

a	1
b	2
c	3
d	4
e	5
f	6
g	7
h	8
i	9
j	10

Hash Code: 421

PATTERN

Size=3

d	b	a
1	2	3



Rabin Karp Matcher(T, P, d, q)

$n = T.length$

$m = P.length$

$h = d^{m-1} \bmod q$

$p = 0$

$t_0 = 0$

for $i = 1$ **to** m

// preprocessing

$p = (dp + P[i]) \bmod q$

$t_0 = (dt_0 + T[i]) \bmod q$

for $s = 0$ **to** $n - m$

// matching

if $p == t_s$

if $P[1..m] == T[s+1..s+m]$

print "Pattern occurs with shift" s

if $s < n - m$

$t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$

Note:



Rabin Karp Algorithm- Usage of Mod Operation

- Hash codes for very long pattern will be huge numbers
- Sometimes beyond system memory limits
- Use a prime number (q) and make all computations in modulo q arithmetic
- So that the numbers will always be $\leq q$
- But the chances of spurious hits increase by this modification



Rabin Karp Algorithm- Analysis

- $O(n + m - 1)$ in average case, if no spurious hits
- In worst case, all can be spurious hits, defines rolling hash function So the worst case complexity is $O(mn)$
- Generally the complexity of Rabin Karp would be between $O(n + m - 1)$ and $O(mn)$

